

推导完全分类

我会先判定 §9.1 的 AncOK 是“过滤约束”还是“生成父仓规则”，再用这个结论检验 depth1/2 为零与 depth≥3 转正是否理论自治。

接下来我会进一步区分父声部“独立入场证书激活”与“由子腿反推激活”的两种模型，并据此判断 §13/§14 差异是否足以解释 depth1/2 被剪。

你的这个问题的核心判定是：

§9.1 的 AncOK 是“约束/过滤器”，不是“生成器”。

所以，严格按 §8/§9 的递归声部树语义，子声部想开仓不能反向强制父声部开仓。父声部必须已经作为 active position 存在，子声部才能通过祖先闭合。买卖点叶子、carrier 结构存在、registry alive，都不等于父声部 active。

但还要补一个关键修正：

父声部不一定必须由“同级裸 BSP”开启，

更准确地说：

父声部必须由“归属于该父 carrier 的 accepted opening certificate”开启。

这个 certificate 可以是本级买卖点，也可以是区间套把低级别终端买卖点包装成的父级开仓证书。低级别 BSP 本身不会自动开父声部；只有当它被解释器包装成 parent-carrier certificate 时，才可以开父声部。

所以最终判定是：

P 的主干是对的：AncOK 不生成父声部，父 active 必须独立进入 raw active set。

但 P 里“必须由该级别自身 BSP”这句话应改成：

必须由该 carrier 自己的 accepted certificate 开启。

而 Q 如果理解为“完整 §13 position instance 会让子级 BSP 自动激活父声部”，则是错的。§13 修的是位置实例身份，不是父声部生成规则。

1. 形式化对象

令结构 carrier 集合为：

$$\mathcal{C}.$$

每个 carrier $c \in \mathcal{C}$ 是一个走势容器、操作容器或 Cell carrier，带有级别：

$$\ell(c).$$

令买卖点/区间套证书集合为：

$$\Gamma_t.$$

每个证书 $\gamma \in \Gamma_t$ 至少有：

$$\kappa(\gamma) \in \mathcal{C}, \quad \sigma(\gamma) \in \{+1, -1\},$$

其中：

$$\kappa(\gamma) = \text{该证书要开启的 carrier},$$

$$\sigma(\gamma) = \text{该证书的开仓方向}.$$

注意： γ 可以由一个裸 BSP 产生，也可以由区间套产生。例如：

$$\gamma = (c; g_\ell, g_{\ell-1}, \dots, g_e; \delta),$$

表示“用低级别终端信号 g_e 证明 carrier c 上的方向 δ 开仓”。

所以真正开仓的是：

$$\gamma \text{ 所属的 carrier } \kappa(\gamma),$$

不是裸买卖点叶子本身。

2. §13 完整 position instance

完整 §13 的 position voice 定义为：

$$v = (c, \gamma, \sigma, n),$$

其中：

$$c = \kappa(\gamma), \quad \sigma = \sigma(\gamma),$$

n 是 generation / nonce / open sequence。

所以：

$$\text{carrier}(v) = c,$$

$$\text{entryCert}(v) = \gamma,$$

$$\text{side}(v) = \sigma.$$

完整 position ID 应为：

$$\text{posId}(v) = H(c, \gamma, \sigma, n).$$

而 §14 简化版相当于把位置身份压缩成：

$$\text{posId}^{\text{simp}}(v) = c$$

或近似只看 carrier。

这个简化会导致多个 entry certificate、方向、generation 被合并，但它不会改变一个基本事实：

$$\text{没有 accepted certificate, carrier 不会自动成为 active position.}$$

3. 父声部进入 A_t 的充要条件

令 $A_t \subseteq \mathcal{V}$ 是最终 active voices 集合。

令 raw active set 为：

$$A_t^{\text{raw}} = H_t^{\text{live}} \cup O_t,$$

其中：

$$H_t^{\text{live}} = \{v : v \text{ 是上一时刻 held, 且未 close / invalidate}\},$$

$$O_t = \{v_\gamma = (\kappa(\gamma), \gamma, \sigma(\gamma), n) : \gamma \in \Gamma_t^{\text{acc}}\}.$$

也就是说：

$$v \in A_t^{\text{raw}} \iff v \text{ 是存活 held position 或 } v \text{ 由本 carrier 的 accepted certificate 开启.}$$

最终 active set 为：

$$A_t = \text{AncOK}^{\text{cont}}(A_t^{\text{raw}}) = \{v \in A_t^{\text{raw}} : \text{Anc}_\mathcal{V}(v) \subseteq A_t^{\text{raw}}\}.$$

因此，对任意 voice v ：

$$v \in A_t \iff v \in A_t^{\text{raw}} \wedge \text{Anc}_V(v) \subseteq A_t^{\text{raw}}.$$

这就是父声部 active 的严格充要条件。

所以问题 1 的答案是：

父声部进入 A_t 的条件不是“任一子级 BSP 触发”。

而是：

父声部本身已经 held-live，或有 accepted certificate 归属于该父 carrier。

如果某个低级别 BSP g 只是 child leaf，并没有被解释器包装成：

$$\gamma = (c; \dots; g; \delta)$$

且满足：

$$\kappa(\gamma) = c,$$

那么它不能开启父 carrier c 。

4. AncOK 是约束，不是生成规则

§9.1:

$$\text{AncOK}(A) = \{v \in A : \text{Anc}(v) \subseteq A\}.$$

立刻有：

$$\text{AncOK}(A) \subseteq A.$$

所以它永远不会添加任何节点。

若子声部 $u \in A$ ，但父声部 $p(u) \notin A$ ，则：

$$u \notin \text{AncOK}(A).$$

也就是说：

$$p(u) \notin A \implies u \text{ 被剪。}$$

这和生成闭包不同。

生成闭包会写成：

$$\text{UpClose}(A) = A \cup \text{Anc}(A).$$

若采用：

$$A_t = \text{UpClose}(A_t^{\text{raw}}),$$

那么子声部会强制父声部进入 active set。

但这不是 §9.1。因为：

AncOK 是最大祖先闭合子集；

UpClose 是最小祖先闭合超集。

二者方向相反：

$$\text{AncOK}(A) \subseteq A \subseteq \text{UpClose}(A).$$

所以问题 3 的答案是：

$a_{u,t} \leq a_{p(u),t}$ 在 §9.1 中是约束，不是生成规则。

它的含义是：

$$a_{p(u),t} = 0 \Rightarrow a_{u,t} = 0.$$

不是：

$$a_{u,t} = 1 \Rightarrow \text{系统强制生成 } a_{p(u),t} = 1.$$

如果实装把它读成“子要开，所以自动开父”，那已经离开 §9.1 语义。

5. §8 的递归 Cell 也不是子生成父

§8：

$$\text{Cell}_{\sigma,\ell} = \text{Local}_{\sigma,\ell} \times \prod_w (1 + \text{Cell}_{-\sigma,\ell_w}).$$

严格解释是：

一个父 Cell 存在后，它可以选择包含若干反向子 Cell。

其中：

$$1 + \text{Cell}_{-\sigma,\ell_w}$$

表示：

该子声部缺席 或 该子声部存在.

但这个乘积是建立在父项：

$$\text{Local}_{\sigma,\ell}$$

已经存在的前提下。

所以：

$\text{Cell}_{-\sigma,\ell_w}$ 的存在依赖父 Cell，

而不是：

$\text{Cell}_{-\sigma,\ell_w}$ 的存在生成父 Cell。

因此问题 2 的答案是：

§8 中子声部的存在应由父 Cell 激活后允许，不是由子声部反向触发父声部激活。

父声部的 $\text{Local}_{\sigma,\ell}$ 需要自己的开仓状态：

$$a_{v,t} = 1.$$

这个状态来自：

held-live 或 carrier-assigned accepted certificate.

6. 对命题 P / Q 的判定

命题 P 的严格版本

P 应写成：

depth- d 子声部 u 要 active，必须有整条 active position 祖先链。

即：

$$u \in A_t \implies \text{Anc}(u) \subseteq A_t.$$

其中父声部 $v = p(u)$ 进入 A_t 的条件是：

$$v \in A_t^{\text{raw}} \wedge \text{Anc}(v) \subseteq A_t^{\text{raw}}.$$

而：

$$v \in A_t^{\text{raw}}$$

当且仅当：

$$v \in H_t^{\text{live}}$$

或：

$$\exists \gamma \in \Gamma_t^{\text{acc}} \quad v = (\kappa(\gamma), \gamma, \sigma(\gamma), n).$$

因此：

父 carrier 仅 registry alive 不够。

必须有：

父 position active。

所以如果中间级 accepted certificate 稀疏，父 position 极少 active，那么 depth1/2 子腿被剪是 §9.1 的正确行为。

命题 Q 的严格判定

Q 说：

active_depth1/2=0 是简化实装产物，完整 §13 后父会 active。

这个命题只有在下面这个条件成立时才对：

实际上已经存在父 carrier 的 accepted certificates，但 §14 简化 ID 导致它们没有被表示为 active pos

例如：

- 同一 carrier 多个 entry signal 被 carrier-id 简化覆盖；
- 多空方向被 carrier-id 合并；
- generation 冲突导致 held parent 无法匹配；
- parent lookup 需要 full posId，但简化版只给 carrier id；
- 完整 $H(c, \gamma, \sigma, n)$ 本可区分出的 parent voice 被 §14 合并或丢弃。

但如果事实是：

父 carrier 只有 registry structure alive，没有 accepted opening certificate，

那么完整 §13 也不会让父 active。

§13 只把：

$$(c, \gamma, \sigma, n)$$

表示成独立 position instance。

它不创造：

$$\gamma.$$

所以：

完整 §13 修的是“身份与多实例表示”，不是“父声部生成规则”。

因此，Q 的强版本是错的。

更准确的判定是：

若父级没有 carrier-assigned accepted certificate,
depth1/2 被剪是 §9.1 必然。

若父级有 accepted certificate 但 §14 简化版没能形成/匹配 parent position,
那才是简化实装产物, 完整 §13 可修。

7. 关键定理：若 $\text{depth} \geq 3$ 通过 AncOK, 则 depth1/2 必须 active

现在看你给的实测：

$$\text{active_depth_ge3} : 0 \rightarrow 14166,$$

但：

$$\text{active_depth1} = 0, \quad \text{active_depth2} = 0.$$

这在严格 §9.1 下有一个非常强的约束。

令：

$$d(v)$$

表示 voice tree 中的真实深度。

设：

$$A_t = \text{AncOK}(A_t^{\text{raw}}).$$

定理

如果：

$$\exists u \in A_t, \quad d(u) \geq 3,$$

那么必然存在：

$$v_1, v_2 \in A_t$$

使：

$$d(v_1) = 1, \quad d(v_2) = 2.$$

因此：

$$\text{active_depth_ge3} > 0 \implies \text{active_depth1} > 0 \wedge \text{active_depth2} > 0.$$

证明

取：

$$u \in A_t, \quad d(u) = d \geq 3.$$

由 AncOK：

$$\text{Anc}(u) \subseteq A_t.$$

u 的祖先链中必然有深度为 1 的祖先 v_1 , 以及深度为 2 的祖先 v_2 。

所以：

$$v_1, v_2 \in A_t.$$

因此 active_depth1 和 active_depth2 均非零。

证毕。

8. 所以 $\text{active_depth} \geq 3 > 0$ 且 $\text{depth1/2} = 0$ 意味着什么？

若你的 active_depth 统计的是同一个 **post-AncOK active set** A_t ，且 depth 是同一声部树里的祖先深度，那么：

$$\text{active_depth_ge3} > 0 \wedge \text{active_depth1} = \text{active_depth2} = 0$$

在数学上不可能。

因此至少有一个条件不成立。

可能性只有几类。

情况 A：depth≥3 统计的不是同一套 A_t

比如：

- $\text{depth} \geq 3$ 统计了 production AncOK 准入后的某类 restored path；
- active_depth1/2 只统计 current snapshot present parents；
- LiveDetached / restored parents 没被计入 active_depth1/2 。

这时不是 §9.1 的结构结论，而是 telemetry 口径不一致。

如果 LiveDetached 父路径确实被 AncOK 用来放行 $\text{depth} \geq 3$ ，那么这些 LiveDetached ancestors 在 §9.1 语义下就属于 active ancestor set。

它们应该被计入某种：

$$\text{active_detached_depth1}, \quad \text{active_detached_depth2}.$$

否则你的 active_depth1/2 不是 A_t 的计数。

情况 B：depth 标签不是 AncOK 的树深度

如果 active_depth_ge3 的 depth 是 signal nesting depth、structural depth、区间套深度，而 active_depth1/2 是 voice-tree depth，那定理不适用。

但此时名称必须区分：

$$\text{signal_depth} \neq \text{voice_depth}.$$

情况 C：production AncOK 检查的是 registry_alive，而不是 active position

你给的数据：

$$\text{sd_parent_registry_alive} \approx 100\%,$$

但：

$$\text{sd_parent_held} = 0.$$

如果实装用：

$$\text{parent carrier registry alive}$$

来代替：

$$\text{parent active position}$$

放行了 $\text{depth} \geq 3$ ，那么这不是 §9.1 的 AncOK。

因为 §9.1 要求的是：

$$a_{u,t} \leq a_{p(u),t},$$

这里 $a_{p(u),t}$ 是 active position，不是 carrier exists in registry。

所以：

$\text{registry_alive} \neq \text{parent_voice_active}.$

若生产 AncOK 用 registry_alive 放行子腿，那是实现语义漂移。

情况 D: depth ≥ 3 不是 ShortDiff 腿

你给的:

$$\text{sd_parent_held} = 0$$

只约束 ShortDiff 子腿。

如果 active_depth ≥ 3 里面大部分不是 ShortDiff，而是别的角色，那么 sd_parent_held=0 不直接矛盾。

但如果 active_depth ≥ 3 正是 ShortDiff 腿，则在 §9.1 下必须有:

$$\text{sd_parent_held} > 0.$$

否则不可能通过。

9. “深层转正、浅层全剪”是不是 PDF 允许?

严格答案:

如果 depth 是同一 voice tree 的深度，PDF §9.1 不允许深层 active 而浅层 ancestor 不 active。

因为祖先闭合定理已经证明:

$$d \geq 3 \text{ active} \Rightarrow d = 1, 2 \text{ ancestors active}.$$

所以:

active_depth ≥ 3 转正而 active_depth $1/2$ 为 0，不是 §9.1 的数学结论。

它只能是:

1. 计数口径不一致;
2. depth 定义不一致;
3. LiveDetached ancestors 被用来放行但未计数;
4. AncOK 实装用 registry_alive 替代 active position;
5. depth ≥ 3 腿并非 ShortDiff;
6. 简化 §14 导致 active parent identity 被隐藏或合并。

这需要用 instrumentation 分辨。

10. §13 完整 position instance 与 §14 简化版的形式差异

§13 完整版

position voice:

$$v = (c, \gamma, \sigma, n).$$

active set:

$$A_t^{13} \subseteq \mathcal{C} \times \Gamma \times \{+1, -1\} \times \mathbb{N}.$$

父关系:

$$p^{13}(u) = v$$

要求:

$$\text{carrier}(v) = \text{par}_C(\text{carrier}(u)),$$

且方向满足对应关系:

$$\sigma(u) = -\sigma(v)$$

对于 ShortDiff。

因此，同一 carrier 上可以有多个 position instance：

$$(c, \gamma_1, +, n_1), \quad (c, \gamma_2, +, n_2), \quad (c, \gamma_3, -, n_3),$$

只要策略允许。

§14 简化版

position identity 近似为：

$$v^{14} = c.$$

active set：

$$A_t^{14} \subseteq \mathcal{C}.$$

这会丢掉：

$$\gamma, \quad \sigma, \quad n.$$

于是可能发生：

1. 多个 entry 被合并；
2. 多空方向冲突；
3. generation 冲突；
4. parent lookup 找不到特定 instance；
5. held parent 被 current carrier 状态覆盖；
6. 不能表达“同一 carrier 上不同证书的不同持仓实例”。

差异是否影响 depth1/2 准入？

只在以下条件下影响：

$$\exists v = (c, \gamma, \sigma, n) \text{ 本应 active, 但 §14 简化后无法表示或无法匹配。}$$

也就是说，完整 §13 会修复：

identity collision / side collision / generation collision / parent-instance lookup failure.

但它不会修复：

$$\text{没有 accepted certificate 产生 parent position。}$$

如果：

$$\Gamma_t^{\text{acc}}(c) = \emptyset$$

且：

$$c \text{ 上没有 held-live position,}$$

那么 §13 和 §14 都不会让 c active。

因此：

$$\text{§13 完整 position instance 是必要的身份修复，但不是父声部自动生成机制。}$$

11. 对 bsp_lvl 稀疏的严格解释

你给的数据：

$$\text{bsp_lvl1} = 8, \quad \text{bsp_lvl} \geq 2 \approx 0.$$

如果实装中 parent voice 只能由对应中间级 raw BSP 开启，那么：

中间级 BSP 稀疏 $\Rightarrow$ 中间级 parent active 稀疏 $\Rightarrow$ depth1/2 ShortDiff 准入稀疏.

这是 P 的自然结论。

但要注意：

正确的 parent opening certificate 不一定等于 raw BSP at same level。

如果区间套允许：

$$\gamma = (c_{\ell+1}; g_{\ell}, g_{\ell-1}, \dots; \delta),$$

那么低级别终端 BSP 可以成为父 carrier 的开仓证书。

这时你应该统计的是：

$$\text{accepted_cert_by_carrier_level}(\ell + 1),$$

而不仅是：

$$\text{raw_bsp_lvl}(\ell + 1).$$

所以判断 P/Q 时，关键不是：

$$\text{bsp_lvl1} \text{ 很少}$$

本身，而是：

parent-carrier accepted certificates 很少。

如果 parent-carrier accepted certificates 也确实接近 0，则 depth1/2 被剪是 §9.1 正确行为。

如果 raw BSP 少，但区间套本应生成 parent-carrier certificate，而实装没生成，那就是证书解释器问题，不是 AncOK 问题。

12. 最深问题：depth1/2 是否本来就该接近 0?

不能从 §9.1 推出。

§9.1 只说：

$$\text{子 active} \Rightarrow \text{父 active}.$$

它不说：

$$\text{浅层应该少, 深层应该多}.$$

在严格自相似的理论中，没有绝对 depth 特权：

同一递归规则在每个相对层级上相同。

所以如果 observed alpha 集中在 depth ≥ 3 ，而 depth1/2 为 0，这不是自相似理论的必然结论。

有三种可能：

可能 1：真实 certificate 分布导致

如果你的 certificate 生成规则确实让浅层 parent active 很稀疏，而深层通过某种 restored persistent path 更常见，那么这是当前 extractor / certificate system 的经验事实。

但这不是 §9.1 逻辑必然。

可能 2：P&L attribution 只归到最深腿

可能浅层 ancestors 是 active carriers，但不被记为 alpha contributors；alpha 只算最深 execution legs。

这时：

$$\text{depth1/2 active count} = 0$$

可能只是 attribution 口径不含 carrier ancestors。

但若它们用于 AncOK，它们应有另一类计数：

$$\text{active_ancestor_depth1/2.}$$

可能 3：实现 bug / telemetry bug

如果深层腿真的通过了 §9.1，而浅层 ancestors 没有 active 计数，那说明：

AncOK 使用的 active set $\neq$ active_depth counters 使用的 active set.

这不是理论结论，而是实现或监控口径不一致。

13. 最终判定表

现象	§9.1 判定	解释
parent carrier registry alive	不够	必须 active position
parent own accepted certificate 存在	可开 parent	可成为 active voice
child BSP 存在	不自动开 parent	除非包装成 parent certificate
AncOK 剪 child	正确	若 parent position 不 active
child 强制 parent active	非 §9.1	这是 UpClose 生成闭包
§13 完整 instance	修身份	不创造证书
depth≥3 active 但 depth1/2=0	不可能	除非计数/深度/AncOK 口径不同

14. 最终结论

严格数学结论是：

§9.1 的 AncOK 是父声部 active 的必要约束。

父声部 active 的充要条件是： held-live 或有归属于该 carrier 的 accepted opening certificate，

子级 BSP 不会自动生成父声部；若要它开父声部，必须先被解释成父 carrier 的 certificate。

完整 §13 position instance 修复的是位置身份、多实例、方向和 generation，不是把 child signal 升格

因此：

P 的主干成立； Q 的强形式不成立。

但是：

$$\text{active_depth}\geq 3 > 0 \wedge \text{active_depth1/2} = 0$$

如果是在同一个 post-AncOK voice active set 上统计，则与 §9.1 矛盾。它不能被解释为 PDF 必然。它说明：

要么 depth 统计口径不同，要么 LiveDetached ancestors 未计数，要么 AncOK 用了 registry alive 而

最后，对 #5 的先验判断：

缠论自相似递归本身不推出 depth1/2 应该接近 0。

如果 depth1/2 真的长期为 0，那要么是 parent-carrier certificates 的生成规则导致，要么是工程口径导致。它不是 §9.1 单独推出的结构定理。

